

Daniel Parker

April 22, 2025

CS 470 Final Reflection

<https://www.youtube.com/watch?v=vaHZ5MECJS0>

Throughout CS 470, I gained hands-on experience building a full-stack, cloud-native web application using AWS services such as API Gateway, Lambda, DynamoDB, and S3. These skills are directly applicable to my professional goals of becoming a back-end or full-stack developer specializing in scalable cloud solutions. The course challenged me to think not just in terms of functionality, but also in terms of security, maintainability, and scalability, which are essential in real-world software development.

Over the span of the course, I developed a deeper understanding of asynchronous workflows, REST API design, and cloud architecture. I also learned how to write and deploy serverless functions, handle IAM roles and policies, and set up CORS and method responses in API Gateway—skills that are in high demand across cloud-centric job roles.

My strengths as a developer lie in problem solving, system architecture, and adaptability. I've worked with both monolithic and serverless architectures, and I'm confident designing efficient, modular systems that are easy to debug and scale. I'm also comfortable working with modern development tools like AWS SDKs, Git, Node.js, and frontend frameworks. Based on these skills and experiences, I'm well prepared to take on an entry-level:

- Cloud Application Developer
- Back-End Developer
- Full-Stack Developer

Understanding how to plan for future growth in the cloud is essential to building long-term, cost-effective applications. Through this course, I've gained practical experience working with serverless technologies and now understand how microservices architecture can improve scalability and resilience. To ensure future growth of the application, I would adopt the following strategies in their respective areas:

Handling Scale and Error Management

- Utilize API Gateway throttling and usage plans to prevent abuse and provide rate limits per client.
- Integrate CloudWatch for centralized logging, enabling real-time monitoring and automated alarms for Lambda errors.
- Separate logic into microservices so that only the impacted functions scale when needed.

Predicting Costs

- AWS provides tools like the AWS Pricing Calculator and Cost Explorer, which I would use to predict expenses based on traffic and usage projections.
- Since Lambda charges based on execution time and number of invocations, I can create predictable cost models by analyzing current usage metrics and simulating higher loads.

Containers vs. Serverless

- Serverless (Lambda) is more cost-efficient and scalable for spiky or unpredictable workloads. It reduces overhead because there's no infrastructure to manage.
- Containers (e.g., ECS or EKS) offer more control and can be cost-predictable with consistent workloads, but require more effort to maintain and scale.
- For my current app, serverless is the better fit due to its low-maintenance nature and scalability. However, I would consider containers if the app grows into a more stateful or complex system that needs custom runtimes or long-lived processes.

Elasticity & Pay-for-Service

- Elasticity ensures the app can scale on demand without over-provisioning resources—important for supporting events like traffic spikes.
- The pay-as-you-go model allows us to minimize costs during low usage and invest only when needed, which is ideal for startups or MVPs.
- As the application grows, I can define auto-scaling policies and reserve capacity selectively for cost optimization.

	Pros	Cons
Serverless	Low cost, automatic scaling, minimal ops overhead	Cold starts, limited runtime duration
Containers	More control, persistent state, custom environments	More complex, higher ops effort
Microservices	Easier to maintain and scale independently	More network latency, complex monitoring

Overall, CS 470 has empowered me with the technical and architectural skills to not only build cloud-native applications but to also think critically about their future growth. With a foundation in serverless architecture, cost modeling, and cloud security, I'm confident in my ability to contribute meaningfully to a cloud development team, and to scale applications efficiently using best practices learned throughout this course.